

Engenharia de Software – Ficha prática 1

Mines Finder - Tutorial de criação de um jogo simples

Objectivos:

- Criar aplicações gráficas interativas
- Utilizar componentes visuais de forma estática (editor visual) e dinâmica (em tempo de execução)
- Criar e personalizar componentes visuais
- Utilizar eventos de forma estática (editor visual) e dinâmica (em tempo de execução)
- Criar eventos próprios
- Persistir informação em ficheiros binários
- Utilizar o padrão Model-View-Controller
- Relembrar conceitos básicos de P.O.O. e de Java

Jogo e Regras

Pretende-se criar um jogo cujo objetivo é descobrir num campo retangular as minas que estão escondidas. Assim, o jogador pode:

- Iniciar um jogo de acordo com um nível de dificuldade (fácil, médio ou difícil)
- Assinalar uma quadrícula como tendo uma mina
- Assinalar uma quadrícula como duvidosa
- Revelar uma quadrícula não marcada da área de jogo. Caso a quadrícula tenha uma mina, o jogador perde o jogo. Caso não tenha uma mina, mas pelo menos uma das 8 quadrículas vizinhas tenha uma mina, é apresentado o número de minas existentes nessa vizinhança. Caso nenhuma das quadrículas vizinhas tenha uma mina, então todas as quadrículas vizinhas não marcadas e não reveladas são reveladas.

O jogo termina quando o jogador perde ou quando todas as quadrículas sem minas forem reveladas. Neste último caso, se o tempo decorrido a concluir o jogo for menor do que nas tentativas anteriores para o mesmo nível de dificuldade, então deve ser pedido o nome do jogador e armazenado esse nome, e o tempo, como recorde atual.

Os níveis de dificuldade do jogo fazem apenas variar a dimensão da área de jogo e o número de minas escondidas. A Tabela 1 mostra a relação entre o nível de dificuldade e estas variáveis.

Tabela 1 - Níveis de dificuldade do Mines Finder

Nível de dificuldade	Largura	Altura	Número de Minas
Fácil	9	9	10
Médio	16	16	40
Difícil	16	30	90

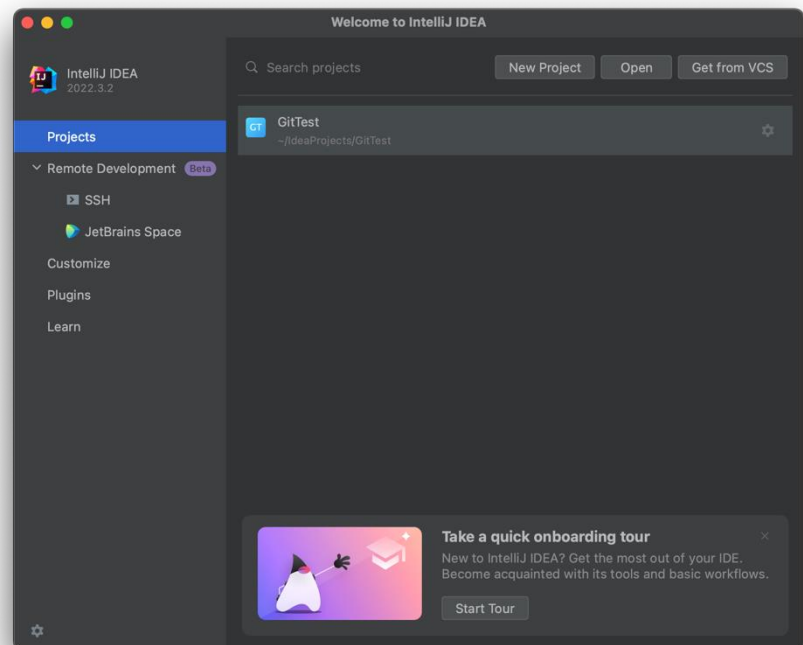
Implementação do jogo

Para a implementação do jogo neste tutorial vamos utilizar a linguagem Java e o IDE [IntelliJ IDEA Ultimate](#). Obviamente, o aluno poderá optar por outro IDE (Eclipse ou Netbeans, por exemplo) ou até por outra linguagem (C#, por exemplo).

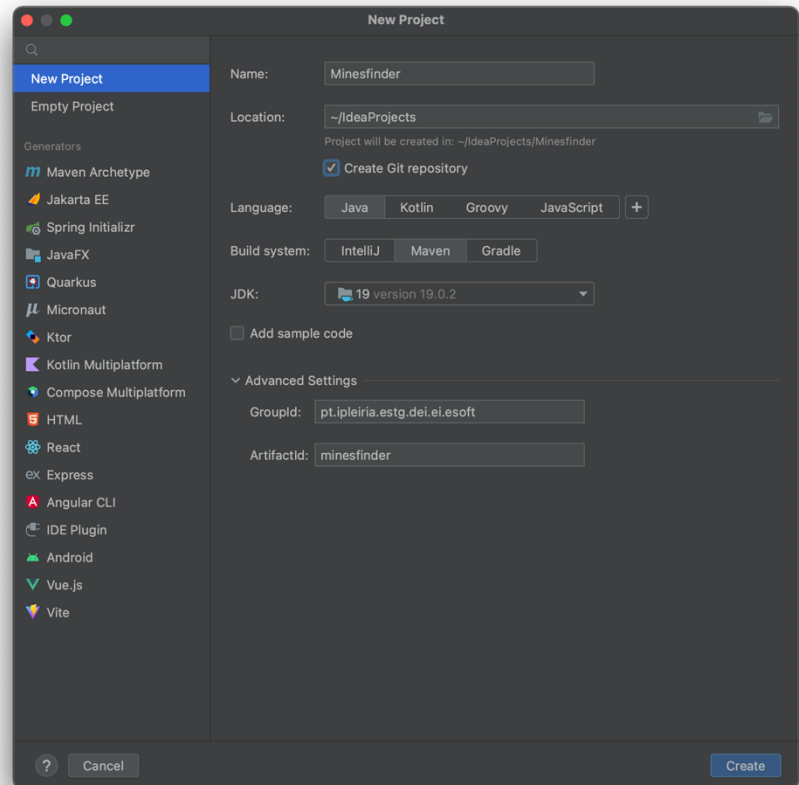
Criação do Projeto

Caso ainda não tenha feito, instale o IntelliJ IDEA (já não há distinção na instalação das várias versões).

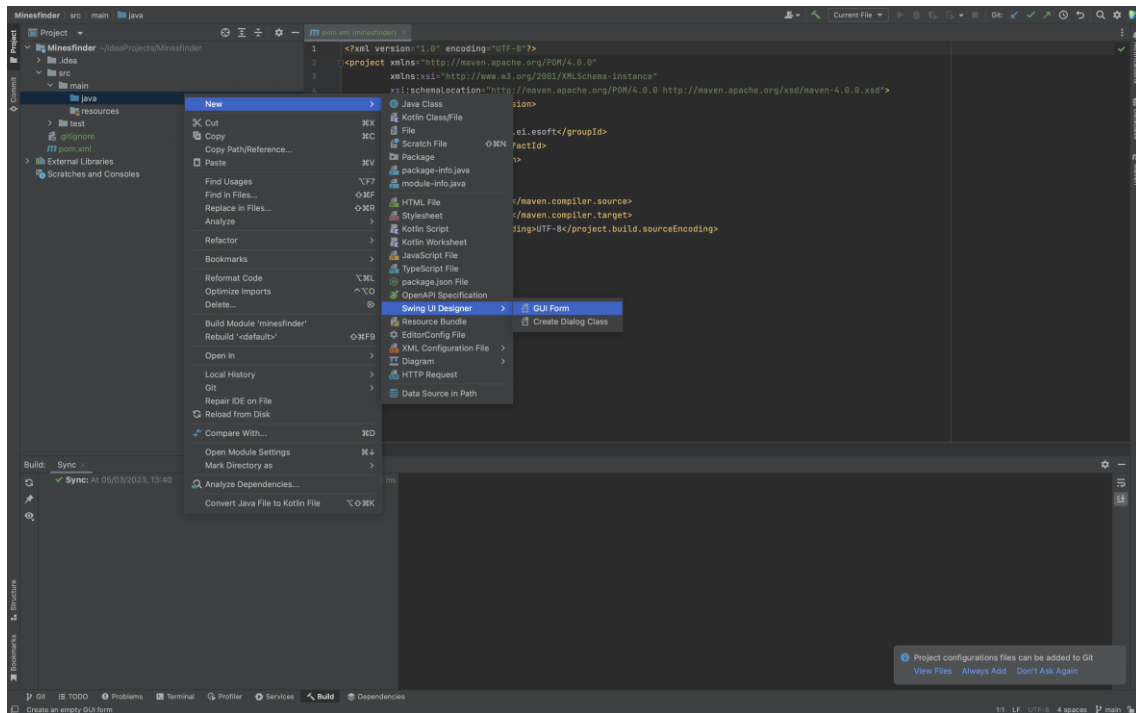
Depois de tudo instalado e configurado, crie um projeto novo (New Project).



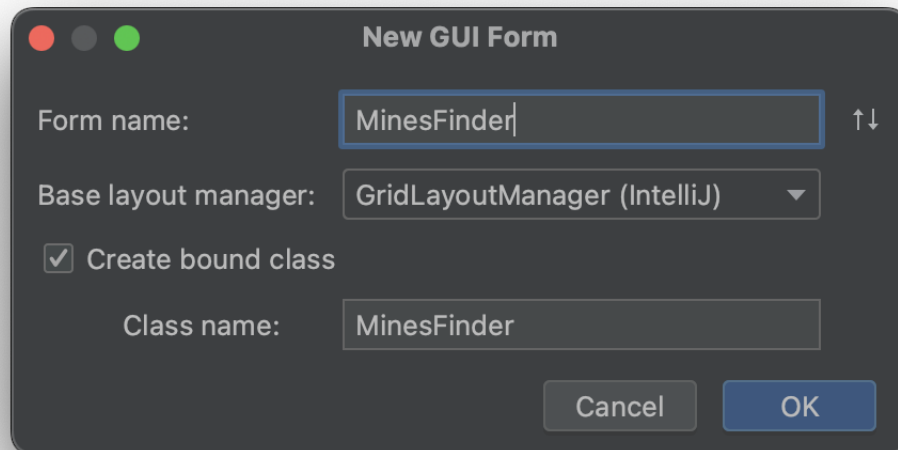
Crie um projeto do tipo Maven, já com o repositório Git criado (Create Git repository) e sem sample code. Insira o group id e o artifact id conforme a imagem.



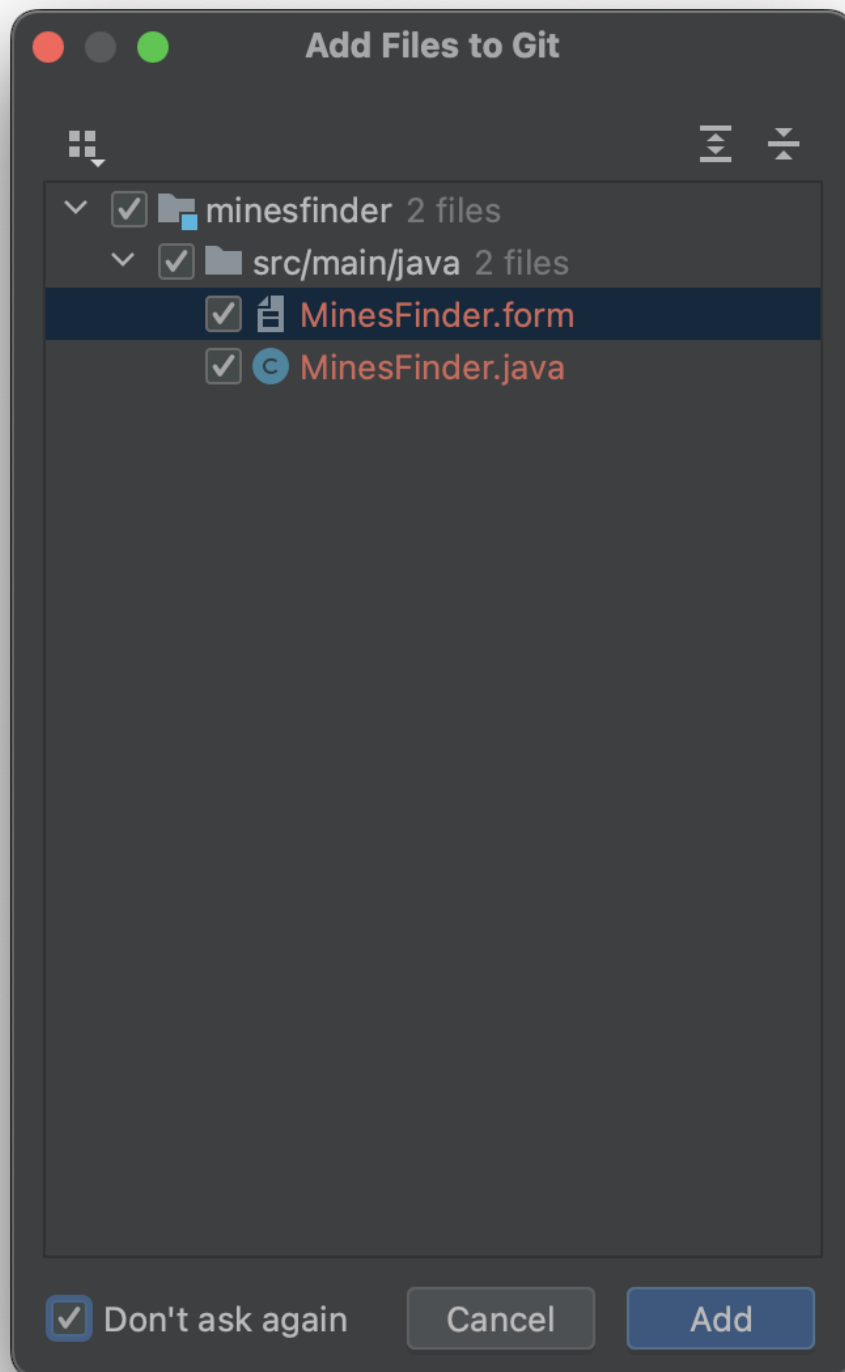
Criação da Janela Principal



No próximo passo, dê o nome “MinesFinder” à classe. Relembrando, em Java todas as classes começam por letras maiúsculas e, quando são compostas por múltiplas palavras, não se utiliza o espaço, mas coloca-se a primeira letra de cada palavra em maiúsculas. Escolha como nome do pacote `pt.ipleiria.estg.es.minesfinder`. O nome de pacote serve para evitar colisão de nomes (duas entidades diferentes a criar classes com os mesmos nomes). É habitual utilizar o domínio internet da entidade “dona” do projeto começando pela parte com maior âmbito para a parte mais específica (basicamente, ao contrário do que é escrito num URL na *web*).



Se o seu IDE sugerir adicionar os ficheiros criados ao sistema de controlo de versão git (comando `git add`) pode optar por adicionar sempre, se preferir. Nas próximas aulas iremos falar sobre.



Ao concluir este *wizard* ficará com a classe criada aberta no editor em modo “design”.

Altere o código da classe MinesFinder para estender de JFrame, por forma a que possa ser instanciável. Crie um construtor que receba o título da janela, e crie a função “main” para instanciar a janela.

```
import javax.swing.*;

public class MinesFinder extends JFrame {

    public MinesFinder(String title) {
        super(title);

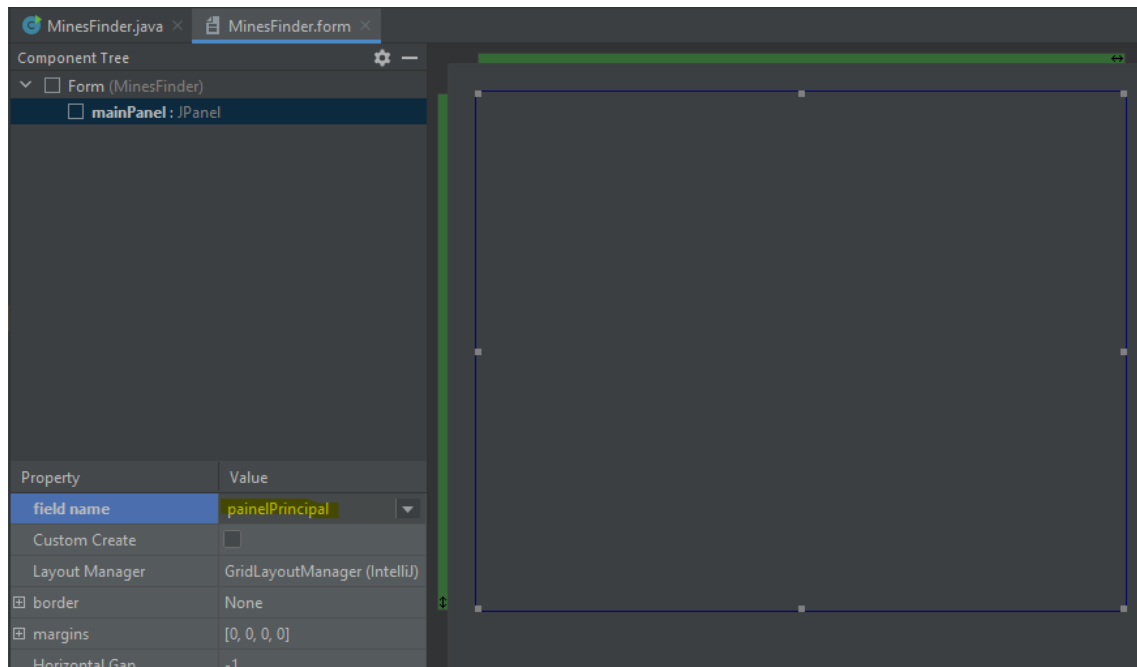
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public static void main(String[] args) {
        new MinesFinder("Mines Finder").setVisible(true);
    }
}
```

Pode executar a aplicação neste momento. Apenas aparecerá uma janela vazia com o título que definiu nas propriedades. Ao fechar essa janela, a aplicação terminará.

Agora vamos adicionar um painel. No modo design, podemos reparar que já existe um painel criado. Só temos de definir o nome da variável para que ele apareça disponível ao programador na classe MinesFinder.

Em “field name” defina o nome “painelPrincipal”.



Agora, na classe MinesFinder, irá aparecer disponível este painel ao programador. Vamos utilizar este painel para o fazer visível na janela.

```

import javax.swing.*;

public class MinesFinder extends JFrame {

    private JPanel painelPrincipal;

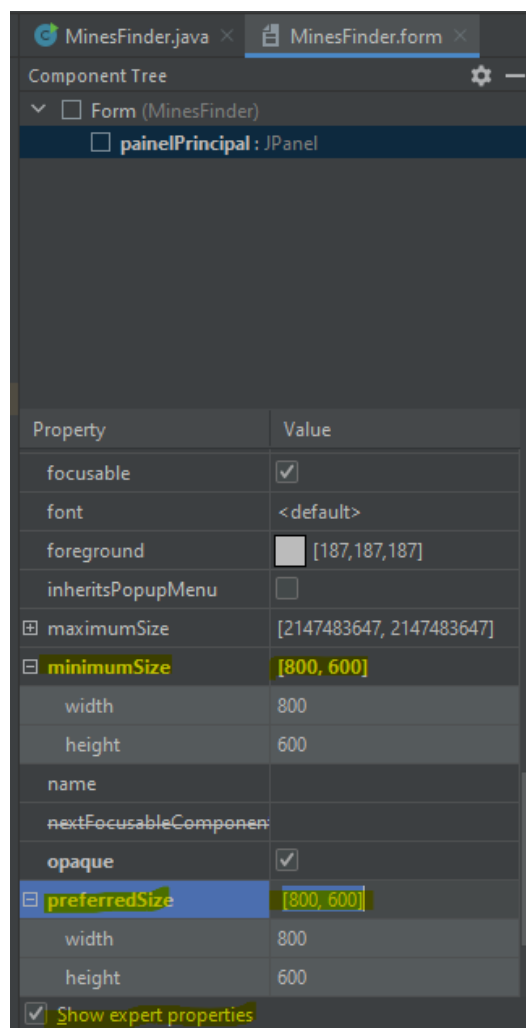
    public MinesFinder(String title) {
        super(title);

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setContentPane(painelPrincipal);
        pack();
    }

    public static void main(String[] args) {
        new MinesFinder("Mines Finder").setVisible(true);
    }
}

```

Em formDesign, ative a checkbox “Show expert properties” e defina os parâmetros “minimum size” e “preferred size” para os valores que considerar suficientes. Sugestão: 800 x 600.

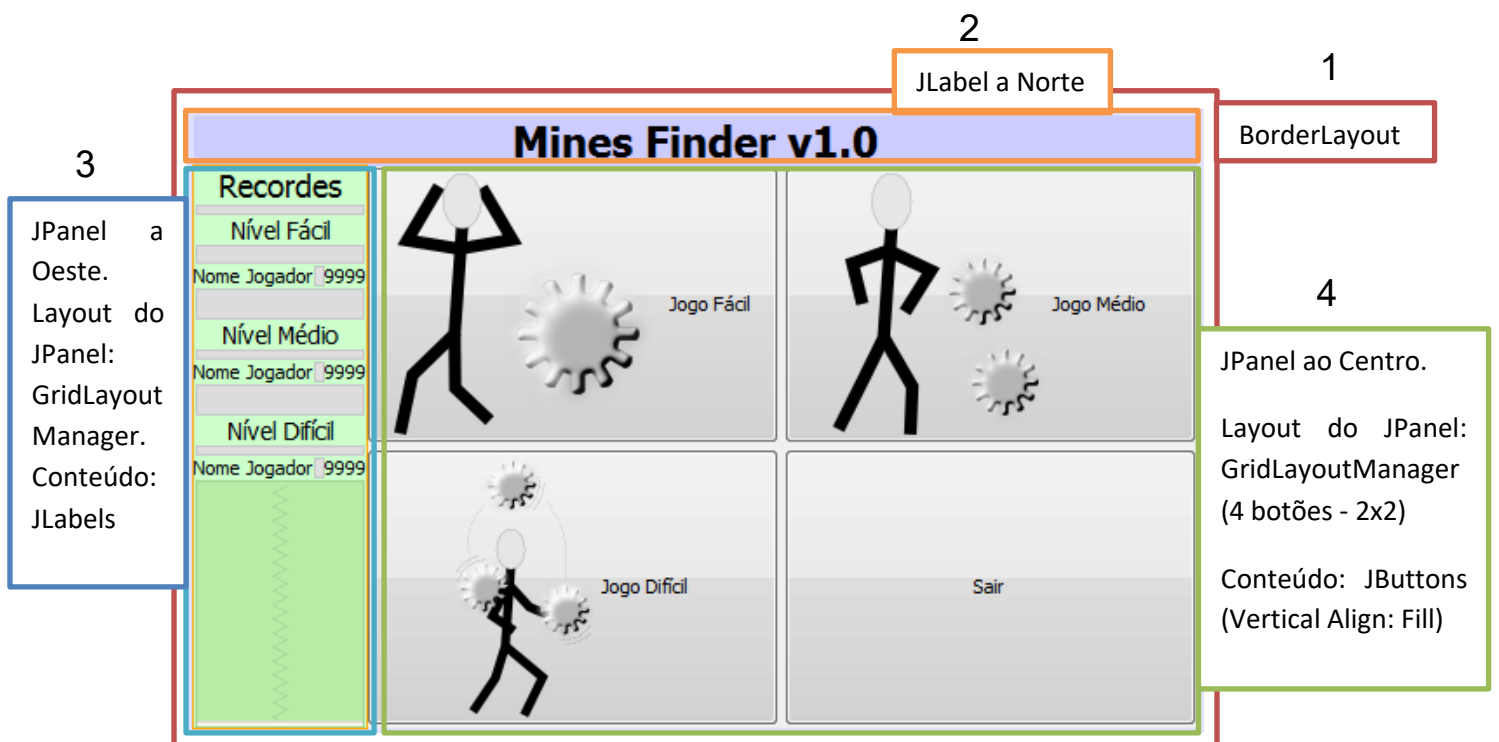


Execute o programa novamente. Irá verificar que já possuímos um painel na janela.

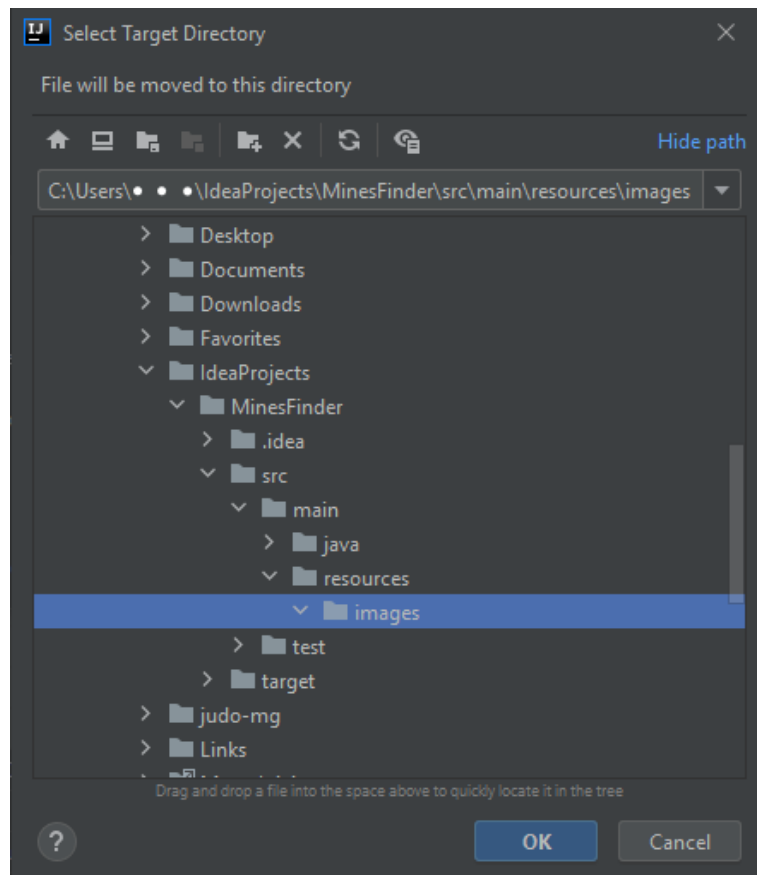
Definição do conteúdo da janela

A linguagem Java foi idealizada como objetivo de se codificar apenas uma vez a aplicação e esta executar-se em diversos sistemas operativos. Assim, uma aplicação Java tanto corre num ambiente Windows, como num ambiente Linux ou Mac OS. Essa independência de ambiente de execução traz também desvantagens, principalmente a nível gráfico. Por exemplo, o aspeto e espaço ocupado por um botão no Windows é diferente de um mesmo botão no macOS. Para lidar com este problema, o Java recorre ao uso de *Layout Managers* (ver Anexo I - Layout Manager).

A janela principal representa um caso em que se têm que utilizar múltiplos Layout Managers. Coloque o seguinte conteúdo dentro da janela principal.



Para colocar imagens nos botões, basta arrastar as imagens para o IDE. Ao arrastar, deve guardar na pasta “src/main/resources/images” (Pode criar a pasta images no explorador de ficheiros ou, ao arrastar, criar a pasta quando lhe aparecer a modal “Select Target Directory”).

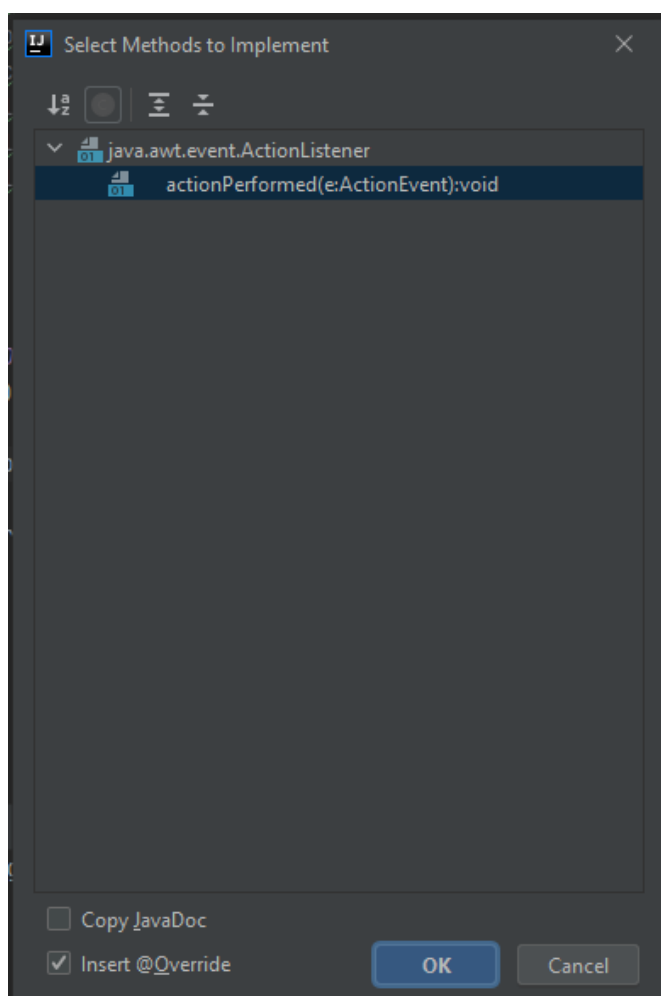
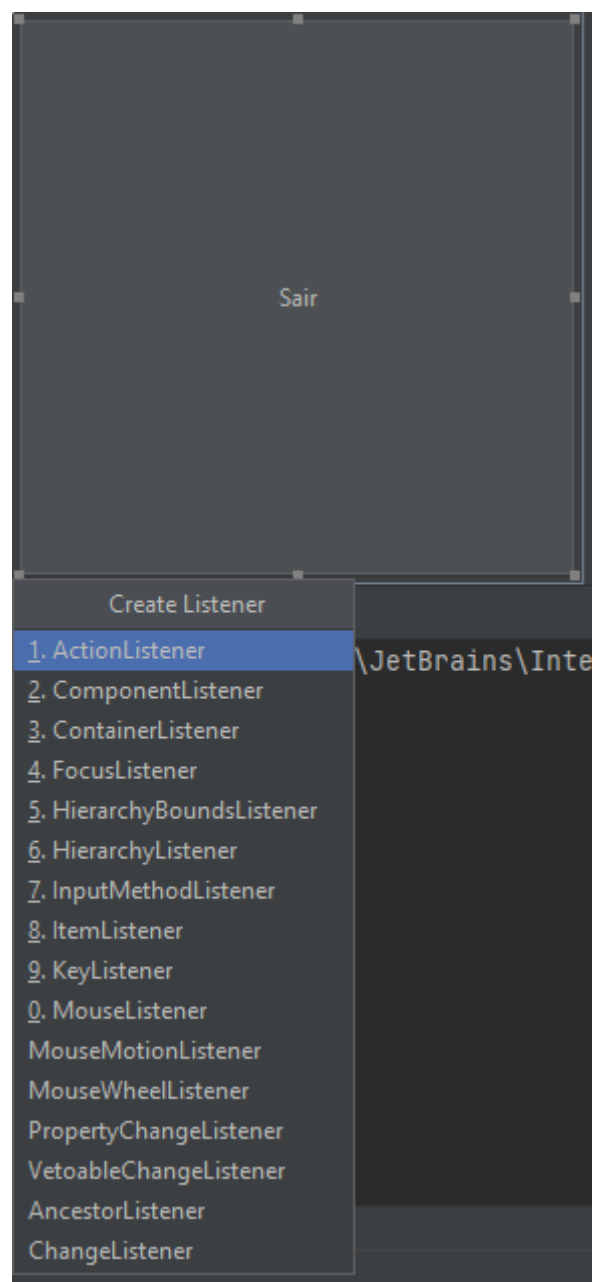
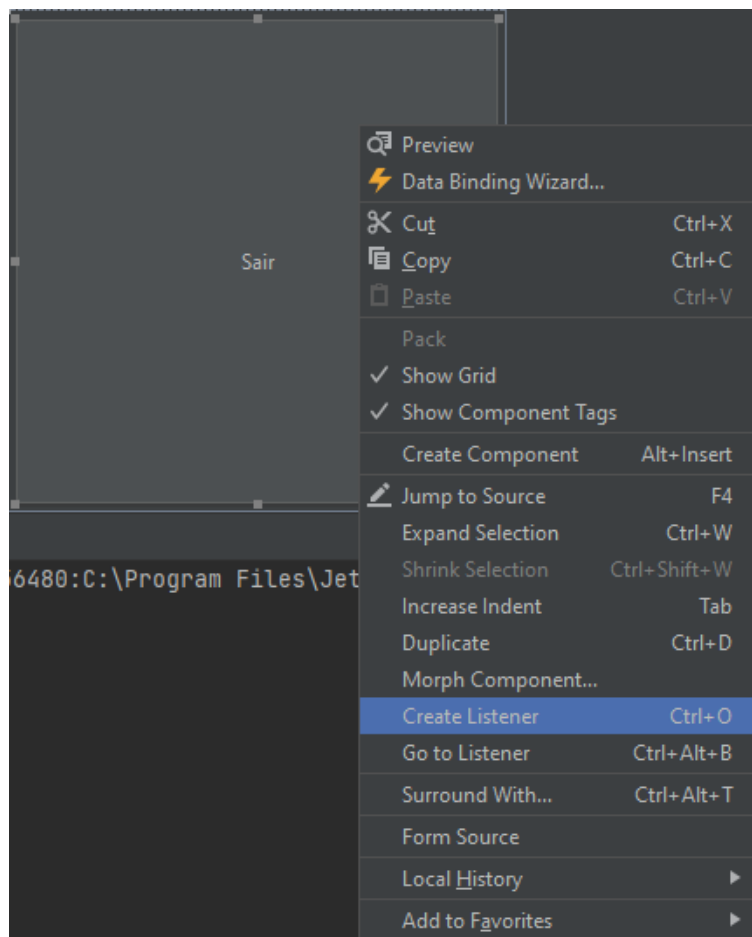


Altere a propriedade “icon” de cada botão seleccionando a imagem que se quer utilizar. Experimente executar o projeto. A janela aparecerá agora com o conteúdo que definimos. Experimente alterar o tamanho dessa janela ainda durante a execução. Vai ver o *Layout Manager* a funcionar, adaptando a posição e o tamanho dos componentes ao espaço disponível.

Resposta a Eventos

Quando o utilizador interage com a aplicação, esta recebe a ação do utilizador em forma de eventos. Cada componente pode responder a um conjunto de eventos, tais como botão do rato pressionado, tecla carregada, entre outros. Na janela principal temos 4 componentes com os quais se espera que o utilizador interaja: os 4 botões. Vamos começar pelo mais simples: o botão “Sair”.

Para programar a resposta a um evento, podemos utilizar o modo “design” do IntelliJ para gerar o código automaticamente. No botão pretendido, clique com o lado direito do rato em “Create Listener” (ou utilizar o atalho sugerido) e escolha o ActionListener (ou clicar no atalho, número 1). Irá reparar que isto não fará nada mais que gerar o código por nós.



Repare que foi gerado o seguinte código no construtor da classe MinesFinder:

```
public MinesFinder(String title) {
    super(title);

    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setContentPane(painelPrincipal);

    // Causes this Window to be sized to fit the preferred size and layouts
    of its subcomponents.
    pack();

    btnSair.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            // TODO
        }
    });
}
```

Poderá escrever manualmente o código dos restantes botões ou, se preferir, continuar a gerar através do modo Design.

Para melhorar visualmente o código, podemos separar a “responsabilidade” da ação do botão “Sair” numa função à parte, e desta forma limpar o construtor, ficando mais visivelmente agradável e seguindo as boas práticas dos padrões de Engenharia de Software:

```
public MinesFinder(String title) {
    super(title);

    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setContentPane(painelPrincipal);

    // Causes this Window to be sized to fit the preferred size and layouts
    of its subcomponents.
    pack();

    btnSair.addActionListener(this::btnSairActionPerformed);
}

private void btnSairActionPerformed(ActionEvent e) {
    System.exit(0);
}
```

No caso do botão “Sair”, o tratamento do evento é muito simples: sair do programa. Para isso basta uma instrução:

```
System.exit(0);
```

Experimente executar novamente a aplicação e verifique se o botão “Sair” termina a aplicação.

Passemos agora ao botão “Jogo Fácil”. O objetivo deste botão é iniciar um jogo fácil, onde deverá aparecer um campo de 9x9 botões e onde o utilizador deve descobrir as 10 minas escondidas.

Crie uma janela nova, cuja classe tem o nome JanelaDeJogo (ver a criação de janelas em

Criação da Janela Principal. Não se esqueça de criar o JPanel painelJogo que servirá de container para o contentPane da JFrame). De seguida, e de volta à janela principal, trate o evento “actionPerformed” do botão “Jogo Fácil” de modo a apresentar essa nova janela quando o botão é ativado.

Para apresentar uma nova janela deve-se primeiro instanciá-la (alocar espaço em memória para ela) através do operador *new*.

```
var janela = new JanelaDeJogo();
```

Depois é só tornar essa janela visível, alterando a sua propriedade “visible” para verdadeiro, ou, se preferir, colocar o código “setVisible(true)” na instanciação da JanelaDeJogo.

```
janela.setVisible(true);
```

Classe JanelaDeJogo:

```
import javax.swing.*;

public class JanelaDeJogo extends JFrame {

    private JPanel painelJogo; // painel do jogo. O nome é definido no modo
    Design, em "field name"

    public JanelaDeJogo() {
        setContentPane(painelJogo);

        // Destrói esta janela, removendo-a completamente da memória.
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);

        setVisible(true); // opcional. Pode optar por fazer depois.
    }
}
```

Construtor e métodos da classe MinesFinder:

```

public MinesFinder(String title) {
    super(title);

    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setContentPane(painelPrincipal);

    // Causes this Window to be sized to fit the preferred size and layouts
    of its subcomponents.
    pack();

    btnSair.addActionListener(this::btnSairActionPerformed);

    btnJogoFacil.addActionListener(this::btnJogoFacilActionPerformed);
    btnJogoMedio.addActionListener(this::btnJogoMedioActionPerformed);
    btnJogoDificil.addActionListener(this::btnJogoDificilActionPerformed);
}

private void btnSairActionPerformed(ActionEvent e) {
    System.exit(0);
}

private void btnJogoFacilActionPerformed(ActionEvent e) {
    new JanelaDeJogo();
}

private void btnJogoMedioActionPerformed(ActionEvent e) {
    // TODO
}

private void btnJogoDificilActionPerformed(ActionEvent e) {
    // TODO
}

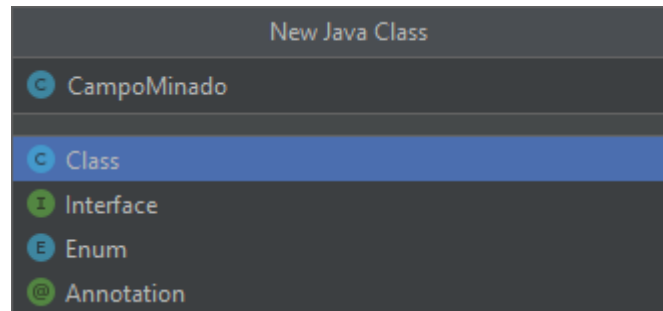
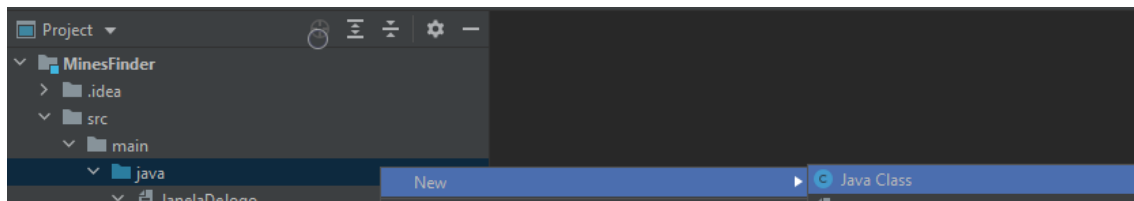
```

Execute o projeto. Repare que quando carrega no botão “Jogo Fácil” aparece uma nova janela, vazia.

Definição do Modelo de Dados

Uma das técnicas mais utilizadas no desenvolvimento de aplicações interativas é a chamada “*Model View Controller*” (MVC). Esta técnica indica que devem ser separadas as responsabilidades das diferentes partes das aplicações. Essas partes correspondem aos dados e seu tratamento (*Model*), a forma como são apresentados (*View*) e como interagem com o utilizador (*Controller*). O Java faz um uso extensivo desta técnica. Assim, a parte visual (*View*) corresponde aos componentes que utilizamos na janela e a parte de interação (*Controller*) corresponde aos métodos que tratam os eventos. Falta-nos, portanto, os dados que vão ser tratados: o *Model*. Vamos então especificar o nosso modelo de dados.

Crie uma nova classe que corresponda ao campo minado. Pode-lhe chamar mesmo CampoMinado. Esta classe será o nosso modelo de dados.



O nosso modelo de dados lida com um campo retangular de $n \times m$ quadrículas (consoante o nível de dificuldade), contendo um número determinado de minas. Acrescente um atributo que guarde se uma qualquer posição desse campo tem ou não uma mina. Um atributo pode ser público (acessível em qualquer parte do código), protegido (acessível apenas na própria classe, suas descendentes e classes que pertençam ao mesmo pacote) ou privado (apenas acessível na própria classe). Devemos sempre tentar manter os atributos das classes o mais restritas possível.

```
private boolean[][] minas;
```

Além da informação sobre onde estão as minas, também precisamos saber se cada quadrícula do campo está tapada ou não, se está assinalada como tendo uma mina, se está assinalada como duvidosa, ou, não tendo uma mina quantas minas existem na vizinhança. Para isso podemos definir uma série de constantes que nos indiquem esses estados.

```
public static final int VAZIO = 0;
/* de 1 a 8 são o número de minas à volta */
public static final int TAPADO = 9;
public static final int DUVIDA = 10;
public static final int MARCADO = 11;
public static final int REBENTADO = 12;
```

Com estas constantes, podemos já definir um atributo que nos guarde o estado de cada quadrícula.

```
private int[][] estado;
```

Precisamos também de saber a largura, altura e número de minas no campo de minas.

```
private int nrLinhas; // ou largura
private int nrColunas; // ou altura
private int nrMinas;
```

Agora que estão definidos os atributos para o CampoMinado, crie um construtor que inicialize esses mesmos atributos. Em Java, os construtores são métodos especiais que não têm tipo de retorno e cujo nome é sempre igual ao nome da classe. São, portanto, os únicos métodos cujo nome deve começar por letra maiúscula.

```

public CampoMinado(int nrLinhas, int nrColunas, int nrMinas) {
    this.nrLinhas = nrLinhas;
    this.nrColunas = nrColunas;
    this.nrMinas = nrMinas;

    this.minas = new boolean[nrLinhas][nrColunas]; // Valores começam a false
    this.estado = new int[nrLinhas][nrColunas]; // Valores começam a 0
}

```

Uma vez que os atributos que definimos são (e bem) privados, crie métodos assessores que permitam determinar o estado de uma quadrícula e se uma quadrícula tem ou não uma mina. Os métodos assessores são métodos cujo objetivo é apenas aceder a atributos privados. Os nomes destes métodos costumam começar por *get*, *is* ou *has* quando os métodos leem o valor do atributo, ou *set* quando definem o valor do atributo. Crie também métodos assessores para obter o número de linhas (largura) e o número de colunas (altura) do campo minado. O código destes dois últimos assessores não está aqui apresentado.

```

public int getEstadoQuadrícula(int x, int y) {
    return estado[x][y];
}

public boolean hasMina(int x, int y) {
    return minas[x][y];
}

```

Definição da Vista

Agora que já temos o modelo de dados e que temos métodos que permitem apurar o estado das quadrículas no campo minado, já é possível representar esses dados na janela. A Tabela 2 mostra como vamos representar o estado das diversas quadrículas utilizando apenas botões.

Tabela 2 - Representação dos diversos estados das quadrículas do campo minado

Estado da quadrícula	Representação	Descrição
Tapada		Botão sem texto
Assinalada como duvidosa		Botão com texto "?" e fundo amarelo
Assinalada como tendo mina		Botão com texto "!" e fundo vermelho
Destapada sem mina e sem minas à volta		Botão sem texto e fundo cinzento escuro
Destapada sem mina e com minas à volta		Botão com o número de minas à volta (1 a 8) como texto e fundo cinzento escuro
Destapada com mina		Botão com texto "*" e fundo cor de laranja

Para manter o código que altera o estado do botão junto do próprio botão, crie um novo tipo de botão: `BotaoCampoMinado`. Este botão estenderá (acrescentará funcionalidades) o `JButton` normal que utilizámos na janela principal. Para isso, crie uma nova classe com nome `BotaoCampoMinado` (ver criação de classes em Definição do Modelo de Dados). Estenda essa classe de `JButton`.

```
public class BotaoCampoMinado extends JButton {  
}
```

Adicione-lhe um atributo para guardar o estado atual da quadrícula e inicialize-o no construtor para o valor TAPADO (ver constantes do CampoMinado em Definição do Modelo de Dados).

```
private int estado;  
  
public BotaoCampoMinado() {  
    this.estado = CampoMinado.TAPADO;  
}
```

Defina agora um método assessor que permita alterar o estado do botão, mudando também o seu aspeto.

```
public void setEstado(int estado) {  
    this.estado = estado;  
    switch (estado) {  
        case CampoMinado.VAZIO:  
            setText("");  
            setBackground(Color.LIGHT_GRAY);  
            break;  
        case CampoMinado.TAPADO:  
            setText("");  
            setBackground(null);  
            break;  
        // Complete o código que falta ...  
        default:  
            setText(String.valueOf(estado));  
            setBackground(Color.LIGHT_GRAY);  
    }  
}
```

Agora que já conseguimos representar uma quadrícula, podemos passar à representação do campo minado inteiro. Uma vez que o campo minado é apenas uma grelha de quadrículas, podemos utilizar um GridLayout para a janela de jogo e colocar lá os botões. Mas lembre-se que para o nível fácil são 9x9 botões, ou seja, 81 botões. No nível difícil são 16x30=480 botões! Colocar esses botões todos à mão não é produtivo. Além disso fica-se sem a noção sobre que botão está em que linha/coluna. Para resolver ambos os problemas, vamos colocar os botões de forma dinâmica (em código) invés de estática (no designer).

Crie um novo atributo na janela de jogo para guardar as referências para os botões e outro para guardar uma referência para o CampoMinado.

```
private BotaoCampoMinado[][] botoes;  
private CampoMinado campoMinado;
```

Altere o construtor de forma a receber o campo minado por parâmetro e inicialize os atributos da classe. Deverá também definir o Layout Manager do painel de conteúdo da janela. Não se esqueça que se só alocar a memória para a matriz de botões, os valores dessa matriz ficam todos a *null*, pelo que deverá criar também os botões e adicioná-los à janela.


```

public JanelaDeJogo(CampoMinado campoMinado) {
    this.campoMinado = campoMinado;

    var nrLinhas = campoMinado.getNrLinhas();
    var nrColunas = campoMinado.getNrColunas();

    this.botoes = new BotaoCampoMinado[nrLinhas][nrColunas];

    painelJogo.setLayout(new GridLayout(nrLinhas, nrColunas));

    // Criar e adicionar os botões à janela
    for (int linha = 0; linha < nrLinhas; ++linha) {
        for (int coluna = 0; coluna < nrColunas; ++coluna) {
            botoes[linha][coluna] = new BotaoCampoMinado();
            painelJogo.add(botoes[linha][coluna]);
        }
    }

    setContentPane(painelJogo);

    // Destrói esta janela, removendo-a completamente da memória.
    setDefaultCloseOperation(DISPOSE_ON_CLOSE);

    // Causes this Window to be sized to fit the preferred size and layouts
    of its subcomponents.
    pack();

    setVisible(true);
}

```

Se experimentar executar o projeto agora, vai verificar que este contém erros. Ao adicionar um parâmetro ao construtor terá que alterar todas as invocações a este mesmo construtor. Volte ao código da janela principal. Repare que o método que trata a ação do botão “Jogo Fácil” terá este código:

```

private void btnJogoFacilActionPerformed(java.awt.event.ActionEvent evt) {
    var janela = new JanelaDeJogo();
    janela.setVisible(true); // se não foi executado no construtor...
}

```

Substitua a instanciação da janela de modo a passar um novo CampoMinado com as dimensões de um nível fácil.

```

private void btnJogoFacilActionPerformed(java.awt.event.ActionEvent evt) {
    var janela = new JanelaDeJogo(new CampoMinado(9,9, 10));
    janela.setVisible(true); // se não foi executado no construtor...
}

```

Agora já pode executar o projeto. Verifique que quando carrega em “Jogo Fácil” aparece a janela de jogo com 81 botões.

Trate agora os eventos de ação para os botões “Jogo Médio” e “Jogo Difícil” (ver Resposta a Eventos) e verifique se na janela de jogo aparecem os botões correspondentes ao nível de dificuldade.

Implementação das Regras do Jogo

As regras do jogo são as ações que se podem fazer durante o jogo. Por outras palavras, são as manipulações possíveis de fazer ao modelo de dados durante um jogo. Em aplicações mais tradicionais, chamam-se a estas ações como regras de negócio.

Um dos conceitos básicos de P.O.O. é o encapsulamento (esconder os dados do exterior das classes). Este conceito implica que os métodos que manipulam dados devam estar na mesma classe que esses dados. Vamos então adicionar métodos à classe `CampoMinado` que permitam e garantam que as regras sejam cumpridas e não violadas.

Regra 1: Quando o jogo começa, todas as quadrículas devem estar tapadas.

Recorde-se que, até ao momento, e em relação ao estado das quadrículas, apenas se alocou memória para a matriz, o que inicializou os valores dessa matriz a 0. Ora, 0 é, de acordo com as constantes que definimos, `VAZIO`. Isto desobedece à regra 1. Para inicializar os valores da matriz estado a `TAPADO`, acrescente o seguinte código ao construtor do `CampoMinado`.

```
...  
  
for (var x = 0; x < nrLinhas; ++x) {  
    for (var y = 0; y < nrColunas; ++y) {  
        estado[x][y] = TAPADO;  
    }  
}
```

Regra 2: O jogador pode revelar quadrículas que estejam tapadas

Se é permitido revelar quadrículas, então deverá existir um método no `CampoMinado` com o nome `revelarQuadrícula()`. Esse método deve garantir que apenas é revelada a quadrícula se esta estiver tapada. Assim, o método poderia ser o seguinte:

```
public void revelarQuadrícula(int x, int y) {  
    if (estado[x][y] < TAPADO) {  
        return;  
    }  
  
    // Faz aqui qualquer coisa...  
}
```

O `if` no código garante que só são reveladas quadrículas cujo estado actual seja `TAPADO`, obedecendo, portanto, à regra.

Regra 3: O jogador nunca pode acertar numa mina à primeira jogada

Para garantir que o jogador nunca acerte numa mina à primeira jogada temos que adiar a colocação das minas pelo campo minado até que o jogador faça a sua primeira jogada. Para isso, acrescente um atributo ao `CampoMinado` chamado `primeiraJogada` e inicialize-o no construtor com o valor lógico verdadeiro.

As minas devem aparecer em posições aleatórias. Crie um método auxiliar (privado) que distribua as minas pelo campo minado, garantindo que não fica nenhuma mina na posição da primeira jogada e que todas as minas ficam em quadrículas diferentes. Para gerar números

aleatórios, precisa utilizar uma instância da classe `Random`. Crie uma variável chamada “aleatório” e inicialize-a.

```
private void colocarMinas(int exceptoX, int exceptoY) {
    var aleatorio = new Random();
    var x = 0;
    var y = 0;

    for (var i = 0; i < numMinas; ++i) {
        do {
            x = aleatorio.nextInt(nrLinhas);
            y = aleatorio.nextInt(nrColunas);
        } while (minas[x][y] || (x == exceptoX && y == exceptoY));

        minas[x][y] = true;
    }
}
```

Finalmente, invoque este método quando uma quadrícula é revelada pela primeira vez.

```
public void revelarQuadrícula(int x, int y) {
    if (estado[x][y] < TAPADO) {
        return;
    }

    if (primeiraJogada) {
        primeiraJogada = false;
        colocarMinas(x, y);
    }

    // TODO...
}
```

Regra 4: Se o jogador revelar uma quadrícula que contém uma mina, o jogo termina e o jogador perde

Esta regra obriga a que seja possível saber se o jogo já terminou ou não e se o jogador perdeu ou não. Para termos esta informação, precisamos acrescentar novos atributos ao nosso modelo de dados. Acrescente dois atributos lógicos com o nome `jogoTerminado` e `jogadorDerrotado`. Inicialize esses atributos a *false* no construtor. Depois, altere o método `revelarQuadrícula()` de forma a que, se a quadrícula a revelar tiver uma mina, o seu estado mude para `REBENTADO`, o jogo fique terminado e com a informação que o jogador perdeu. Para poder comunicar essa informação para o exterior da classe `CampoMinado`, crie dois métodos assessores para os atributos que criou (ver Definição do Modelo de Dados para se recordar sobre métodos assessores). Poderá chamar-lhes `isJogoTerminado()` e `isJogadorDerrotado()`.

Finalmente, esta regra também implica que se o jogo estiver terminado, não deverá ser possível revelar novas quadrículas. Para isso altere o *if* do `revelar quadrícula`, acrescentando esta verificação.

```

public void revelarQuadrícula(int x, int y) {
    if (jogoTerminado || estado[x][y] < TAPADO) {
        return;
    }

    // TODO: continue here
}

```

Regra 5: Se o jogador revelar uma quadrícula que não tem uma mina, deve ser apresentado o número de minas à volta dessa quadrícula

Para apresentar o número de minas à volta de uma quadrícula é preciso contá-las. Crie um método que conte o número de minas à volta de uma quadrícula. Não se esqueça dos casos especiais em que o número de vizinhos é menor: quando a quadrícula está encostada a um extremo. Chame a esse método contarMinasVizinhas(). Não se esqueça que esse método é um método auxiliar e, como tal, não deve estar disponível às outras classes.

```

private int contarMinasVizinhas(int x, int y) {
    var numMinasVizinhas = 0;

    for (var i = Math.max(0, x - 1); i < Math.min(nrLinhas, x + 2); ++i) {
        for (var j = Math.max(0, y - 1); j < Math.min(nrColunas, y + 2); ++j)
        {
            if (minas[i][j]) {
                ++numMinasVizinhas;
            }
        }
    }

    return numMinasVizinhas;
}

```

Com o auxílio deste método, altere o revelarQuadrícula() de forma a alterar o estado da quadrícula revelada para VAZIO (caso não tenha nenhuma mina à volta) ou para o número correspondente ao número de minas à volta.

Regra 6: Se o jogador revelar uma quadrícula que não tem mina e não tem minas à sua volta, as quadrículas à sua volta devem ser automaticamente reveladas

Para garantir esta regra crie um método auxiliar: revelarQuadrículasVizinhas(). Este método deve revelar as quadrículas que estão à volta da localização dada. Não precisa de se preocupar se essas quadrículas estão marcadas ou duvidosas, ou até mesmo se já foram reveladas, porque a regra 2 já foi garantida anteriormente. Invoque este método quando muda o estado de uma quadrícula para VAZIO.

Regra 7: Quando todas as quadrículas que não têm minas forem reveladas, o jogo termina e o jogador ganha

Crie um método auxiliar que verifique se todas as quadrículas que não têm minas estão reveladas. Pode chamar a esse método isVitoria().

```
private boolean isVitoria() {
    for (int i = 0; i < nrLinhas; ++i) {
        for (var j = 0 ; j < nrColunas; ++j) {
            if (!minas[i][j] && estado[i][j] >= TAPADO) {
                return false;
            }
        }
    }
    return true;
}
```

Utilize este método cada vez que revelar uma quadrícula, atribuindo valores aos atributos `jogoTerminado` e `jogadorDerrotado` quando necessário.

Regra 8: O jogador pode marcar uma quadrícula que ainda não foi revelada como tendo uma mina

Crie um método `marcarComoTendoMina()` que altere o estado de uma quadrícula que ainda não foi revelada (ou seja, que o seu estado seja `TAPADO` ou `DUVIDA`) para `MARCADO`.

Regra 9: O jogador pode marcar uma quadrícula que ainda não foi revelada como suspeita de ter uma mina

Crie um método `marcarComoSuspeita()` que altere o estado de uma quadrícula que ainda não foi revelada (ou seja, que o seu estado seja `TAPADO` ou `MARCADO`) para `DUVIDA`.

Regra 10: O jogador pode desmarcar uma quadrícula que ainda não foi revelada de qualquer marca feita anteriormente

Crie um método `desmarcarQuadrícula()` que altere o estado de uma quadrícula marcada (ou seja, que o seu estado seja `MARCADO` ou `DUVIDA`) para `TAPADO`.

Regra 11: Deve ser contabilizado o tempo de jogo

Para cumprir esta regra, vamos assumir que o jogo começa quando a primeira quadrícula é revelada e termina quando o jogador perde ou ganha. Para calcular a duração do jogo, precisamos de guardar o instante em que o jogo começou de forma a podermos calcular o tempo que passou até ao jogo terminar. Para isso crie mais dois atributos: `instanteInicioJogo` e `duracaoJogo`.

```
private long instanteInicioJogo;
private long duracaoJogo;
```

Para obter o instante atual pode utilizar o método estático `currentTimeMillis()` da classe `System`. Este método devolve o número de milissegundos que passaram desde as 00h00 de 1 de Janeiro de 1970 UTC (Tempo Universal Coordenado). Altere o método `revelarQuadrícula()` de forma a que este guarde o instante em que o jogo começou.

```

public void revelarQuadrícula(int x, int y) {
    if (jogoTerminado || estado[x][y] < TAPADO) {
        return;
    }

    if (primeiraJogada) {
        primeiraJogada = false;
        colocarMinas(x, y);

        instanteInicioJogo = System.currentTimeMillis();
    }

    // . . .
}

```

Altere também os locais onde se determina que o jogo terminou de forma a calcular a duração do jogo.

```

public void revelarQuadrícula(int x, int y) {
    // . . .

    if (minas[x][y]) {
        estado[x][y] = REBENTADO;
        jogoTerminado = true;
        jogadorDerrotado = true;
        duracaoJogo = System.currentTimeMillis() - instanteInicioJogo;
    } else {
        // . . .

        if (isVitoria()) {
            jogoTerminado = true;
            jogadorDerrotado = false;
            duracaoJogo = System.currentTimeMillis() - instanteInicioJogo;
        }

        // . . .
    }

    // . . .
}

```

Crie ainda um método assessor que divulgue a duração do jogo para o exterior. Lembre-se que o valor do atributo `duracaoJogo` só é válido se o jogo já terminou. Caso o jogo ainda não tenha terminado, mas já tenha começado, o método pode devolver a duração atual.

```
public long getDuracaoJogo() {  
    if (primeiraJogada) {  
        return 0;  
    }  
  
    if (!jogoTerminado) {  
        return System.currentTimeMillis() - instanteInicioJogo;  
    }  
  
    return duracaoJogo;  
}
```

Definição do Controlador

Neste momento temos as regras do jogo todas definidas, mas ainda não há forma de o jogador realizar essas operações. Isso deve-se ao facto de ainda não termos definido nenhum controlador na JanelaDeJogo. Vamos então defini-lo, começando pela ação dos botões quando pressionados. Cada vez que um botão é pressionado, devemos revelar a quadrícula correspondente a esse botão. Para isso, precisamos definir o método que é invocado pelo evento `ActionPerformed`. Infelizmente, não podemos definir esse método do modo “design” da janela (como fizemos em Resposta a Eventos) porque os botões são criados apenas em tempo de execução e não no desenho da janela. Temos então que definir esse método por código. Para isso, altere o construtor da JanelaDeJogo de forma a incluir um `ActionListener`. O `ActionListener` é uma interface que define o método invocado pelo evento `ActionPerformed`.

```

public class JanelaDeJogo extends JFrame {

    private JPanel painelJogo;

    private BotaoCampoMinado[][] botoes;
    private CampoMinado campoMinado;

    public JanelaDeJogo(CampoMinado campoMinado) {
        this.campoMinado = campoMinado;

        var nrLinhas = campoMinado.getNrLinhas();
        var nrColunas = campoMinado.getNrColunas();

        this.botoes = new BotaoCampoMinado[nrLinhas][nrColunas];

        painelJogo.setLayout(new GridLayout(nrLinhas, nrColunas));

        for (int linha = 0; linha < nrLinhas; ++linha) {
            for (int coluna = 0; coluna < nrColunas; ++coluna) {
                botoes[linha][coluna] = new BotaoCampoMinado(linha, coluna);
                botoes[linha][coluna].addActionListener(this::btnCampoMinadoActionPerformed);
                painelJogo.add(botoes[linha][coluna]);
            }
        }

        setContentPane(painelJogo);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);

        pack();

        setVisible(true);
    }

    public void btnCampoMinadoActionPerformed(ActionEvent e) {
        var botao = (BotaoCampoMinado) e.getSource();

        int x = ???
        int y = ???

        campoMinado.revelarQuadrícula(x, y);
    }
}

```

Repare nas linhas marcadas a vermelho. Como podemos saber qual é a linha e a coluna correspondente ao botão? Este é um dos problemas de avançar com código antes de pensar no problema. Felizmente há várias formas de resolver este problema, mas vamos optar pela mais simples: o próprio botão deveria saber. Para que ele saiba a que linha e coluna corresponde no campo minado, acrescente ao `BotaoCampoMinado` dois novos atributos: `linha` e `coluna`. Inicialize-os no construtor com valores recebidos como parâmetros e crie métodos assessores para estes atributos.


```

public class BotaoCampoMinado extends JButton {
    private int estado;

    private int linha;
    private int coluna;

    public BotaoCampoMinado(int linha, int coluna) {
        this.estado = CampoMinado.TAPADO;

        this.linha = linha;
        this.coluna = coluna;
    }

    public int getLinha() {
        return linha;
    }

    public int getColuna() {
        return coluna;
    }
}

```

Agora que tem esta informação, já pode voltar a alterar o construtor da JanelaDeJogo para a utilizar.

```

for (int linha = 0; linha < nrLinhas; ++linha) {
    for (int coluna = 0; coluna < nrColunas; ++coluna) {
        botoes[linha][coluna] = new BotaoCampoMinado(linha, coluna);
        botoes[linha][coluna].addActionListener(
            this::btnCampoMinadoActionPerformed
        );
        painelJogo.add(botoes[linha][coluna]);
    }
}

```

```

public void btnCampoMinadoActionPerformed(ActionEvent e) {
    var botao = (BotaoCampoMinado) e.getSource();

    var x = botao.getLinha();
    var y = botao.getColuna();

    campoMinado.revelarQuadrícula(x, y);
}

```

Experimente agora executar a aplicação. Quando carrega num botão acontece alguma coisa? Não? Porquê?

Na realidade, quando se carrega num botão, a quadrícula correspondente é revelada, mas o estado dos botões que representam essas quadrículas não é atualizado. Vamos então atualizar

o estado dos botões cada vez que se pressiona num botão. Para isso, precisamos de um método auxiliar: `atualizarEstadoBotoes()`.

```
private void atualizarEstadoBotoes() {
    for (int x = 0; x < campoMinado.getNrLinhas(); x++) {
        for (int y = 0; y < campoMinado.getNrColunas(); y++) {
            botoes[x][y].setEstado(campoMinado.getEstadoQuadrícula(x, y));
        }
    }
}
```

Invoque este método no `actionPerformed()` dos botões. Volte a executar o código. Já funciona? Ótimo. Próximo passo: verificar se o jogo terminou. Depois de atualizar o estado dos botões, verifique se o jogo já terminou. Se for esse o caso, verifique se o jogador ganhou ou perdeu e apresente a mensagem correspondente. No fim do utilizador ler essa mensagem, feche a janela de jogo.

```
public void actionPerformed(ActionEvent e) {
    ...
    atualizarEstadoBotoes();
    if (campoMinado.isJogoTerminado()) {
        if (campoMinado.isJogadorDerrotado())
            JOptionPane.showMessageDialog(null, "Oh, rebentou uma mina",
            "Perdeu!", JOptionPane.INFORMATION_MESSAGE);
        else
            JOptionPane.showMessageDialog(null, "Parabéns. Conseguiu
            descobrir todas as minas em "+
            (campoMinado.getDuracaoJogo()/1000)+" segundos",
            "Vitória", JOptionPane.INFORMATION_MESSAGE);
            setVisible(false);
    }
    ...
}
```

Pode experimentar novamente e tentar ganhar um jogo e perder outro para verificar se as mensagens aparecem corretamente.

Próximo passo: marcar as quadrículas. Para marcar as quadrículas, o jogador deverá pressionar o botão direito do rato sobre a quadrícula que deseja marcar. Se essa quadrícula estava tapada, o jogo marcará essa quadrícula como tendo uma mina. Caso já tivesse marcada como tendo uma mina, o jogo marcará essa quadrícula como sendo suspeita de ter uma mina. Caso já tivesse marcada como sendo suspeita, o jogo retirará a marcação da quadrícula.

Para implementar esta interação com o utilizador, precisamos definir mais um evento: o `mousePressed`. Para isso, precisamos implementar um outro Listener, o `MouseListener`, e adicioná-lo ao botão. Note que a interface `MouseListener` define o método para quando o rato é pressionado, mas também define outros métodos correspondentes a outros eventos relacionados com o rato, pelo que temos que definir esses métodos também, mas podemos deixá-los vazios.

```

...
MouseListener mouseListener=new MouseListener() {
    @Override
    public void mouseClicked(MouseEvent e) {
    }

    @Override
    public void mousePressed(MouseEvent e) {
        if (e.getButton() != MouseEvent.BUTTON3) {
            return;
        }

        var botao = (BotaoCampoMinado) e.getSource();

        var x = botao.getColuna();
        var y = botao.getLinha();

        var estadoQuadricula = campoMinado.getEstadoQuadricula(x, y);

        if (estadoQuadricula == CampoMinado.TAPADO) {
            campoMinado.marcarComoTendoMina(x, y);
        } else if (estadoQuadricula == CampoMinado.MARCADO) {
            campoMinado.marcarComoSuspeita(x, y);
        } else if (estadoQuadricula == CampoMinado.DUVIDA) {
            campoMinado.desmarcarQuadricula(x, y);
        }
        atualizarEstadoBotoes();
    }

    @Override
    public void mouseReleased(MouseEvent e) {
    }

    @Override
    public void mouseEntered(MouseEvent e) {
    }

    @Override
    public void mouseExited(MouseEvent e) {
    }
};

...
botoes[linha][coluna].addActionListener(this::btnCampoMinadoActionPerformed);
botoes[linha][coluna].addMouseListener(mouseListener);
...

```

Execute o projeto e verifique que já consegue marcar quadrículas. Neste momento já é possível executar todas as operações permitidas ao jogador, mas apenas utilizando o rato. Quem quiser utilizar apenas o teclado terá sérias dificuldades em selecionar a quadrícula e é impossível marcá-la como suspeita ou como tendo mina. Vamos então acrescentar uma forma de o utilizador poder jogar apenas com o teclado. Para isso, precisamos de definir mais um evento: o `keyPressed`. Tal como nos casos anteriores, basta implementar uma interface, o `KeyListener` neste caso, e adicioná-la ao botão. Neste evento deveremos permitir escolher outro botão utilizando as setas do teclado e marcar uma quadrícula utilizando a tecla M.

```

...
var keyListener = new KeyListener() {
    @Override
    public void keyTyped(KeyEvent e) {
    }

    @Override
    public void keyPressed(KeyEvent e) {
        var botao = (BotaoCampoMinado) e.getSource();

        var x = botao.getLinha(); // ou var linha = botao.getLinha();
        var y = botao.getColuna(); // ou var coluna = botao.getColuna();

        /*
        NOTA: Usámos as variáveis com nomes x, y para ser mais resumido no PDF, mas esta não é uma boa
        prática. Prefira os nomes linha, coluna sempre que puder, para não confundir e introduzir
        possíveis bugs. Isto porque na mente humana utilizamos muito o x na matemática para o eixo
        horizontal e o y para o eixo vertical. Visto que em programação não existe essa associação,
        sendo até o contrário (como é o presente caso abaixo), torna-se fácil cometer erros.
        Já os nomes linha, coluna nunca confundem o programador e sabemos perfeitamente em que situação
        incrementar cada variável respetivamente.
        Por exemplo: Ao subir com o teclado (VK_UP) o x decrementa um valor. Isto é estranho porque
        visualizamos o x no eixo horizontal a andar para a esquerda (sendo isso o VK_LEFT). Mas se o
        código estiver escrito da seguinte forma:
        case KeyEvent.VK_UP -> botoes[--linha < 0 ? nrLinhas - 1 : linha][coluna].requestFocus();
        Agora o código é mais intuitivo e menos propenso a erros.
        */

        switch (e.getKeyCode()) {
            case KeyEvent.VK_UP -> botoes[--x < 0 ? nrLinhas - 1 :
x][y].requestFocus();
            case KeyEvent.VK_DOWN -> botoes[(x + 1) %
nrLinhas][y].requestFocus();
            case KeyEvent.VK_LEFT -> botoes[x][--y < 0 ? nrColunas - 1 :
y].requestFocus();
            case KeyEvent.VK_RIGHT -> botoes[x][(y + 1) %
nrColunas].requestFocus();
            case KeyEvent.VK_M -> {
                switch (campoMinado.getEstadoQuadrícula(x, y)) {
                    case CampoMinado.TAPADO -> campoMinado.marcarComoTendoMina(x, y);
                    case CampoMinado.MARCADO -> campoMinado.marcarComoSuspeita(x, y);
                    case CampoMinado.DUVIDA -> campoMinado.desmarcarQuadrícula(x, y);
                }
                atualizarEstadoBotoes();
            }
        }

        @Override
        public void keyReleased(KeyEvent e) {
        }
    };
...
botoes[linha][coluna].addMouseListener(mouseListener);
botoes[linha][coluna].addKeyListener(keyListener);
...

```

Repare no código assinalado a vermelho. Este código é exatamente igual ao que está no evento mousePressed. Para não termos código repetido, crie um método auxiliar (marcarQuadrícula())

com este código e substitua-o pela invocação desse método (em ambos os *listeners*). Depois, experimente executar o projeto e jogar um jogo utilizando apenas o teclado.

Para completar o Mines Finder, falta ainda manter e guardar a tabela de recordes. Nesta secção vamos definir o modelo de dados que guarda essa informação, atualizar a vista (que já fizemos em

Criação da Janela Principal) quando esses dados são alterados e garantir que as pontuações não são perdidas quando a aplicação termina.

Definição do Modelo de Dados

Crie uma classe chamada `TabelaRecordes`. Crie atributos para guardar apenas o nome do jogador e o tempo de jogo e inicialize esses atributos no construtor com os valores “Anónimo” para o nome e “9999999” para o tempo (não se esqueça que o tempo está contabilizado em milissegundos). Defina também os métodos assessores de leitura (getters) para esses atributos. Elabore também um método `setRecorde()` que receba o nome e o tempo de um jogador. Lembre-se que um recorde só pode ser atualizado se o tempo for menor que o registado anteriormente. Para guardarmos os recordes para os diferentes níveis de dificuldade, podemos utilizar diversas instâncias desta classe.

Com o modelo de dados completo, podemos começar a utilizá-lo. Para isso, vamos modificar o comportamento quando o jogador alcança uma vitória. Quando o jogador ganha, o jogo deve verificar se o tempo que demorou é menor do que o recorde atual e, se assim for, pedir o nome ao jogador e guardar essa informação nos recordes.

Ainda antes de atualizar a tabela de recordes, precisa de ter acesso a ela na `JanelaDeJogo`. Altere o construtor dessa classe de forma a receber a tabela de recordes e a guardar essa tabela num atributo.

```
...
private TabelaRecordes recordes;

public JanelaDeJogo(CampoMinado campo, TabelaRecordes tabela) {
    this.campoMinado = campo;
    this.recordes = tabela;
}
...
```

Corrija os erros causados por esta alteração do construtor na janela principal. Para isso, na janela principal, deverá criar também três atributos para as tabelas de recordes (`recordesFacil`, `recordesMedio` e `recordesDifícil`) e inicializá-los no construtor. Depois deverá passar esses atributos para a `JanelaDeJogo` quando inicia um jogo fácil, médio e difícil.

Finalmente, tendo acesso à tabela de recordes, altere o comportamento da vitória na `JanelaDeJogo` conforme especificado atrás.

```

...
JOptionPane.showMessageDialog(null, "Parabéns. Conseguiu descobrir todas as
minas em "
    + (campoMinado.getDuracaoJogo() / 1000) + " segundos", "Vitória",
JOptionPane.INFORMATION_MESSAGE);

boolean novoRecorde=campoMinado.getDuracaoJogo()<recordes.getTempo();

if (novoRecorde) {
    String nome=JOptionPane.showInputDialog("Introduza o seu nome");
    recordes.setRecorde(nome, campoMinado.getDuracaoJogo());
}
...

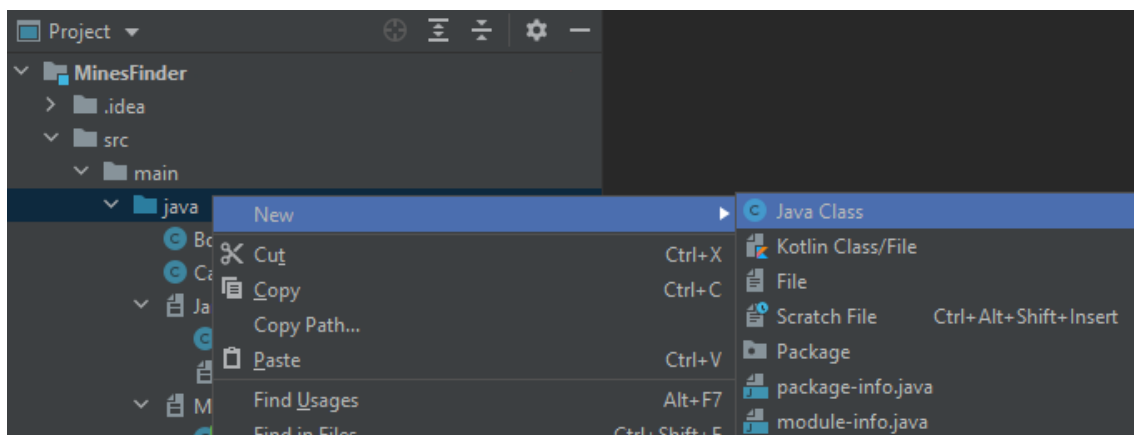
```

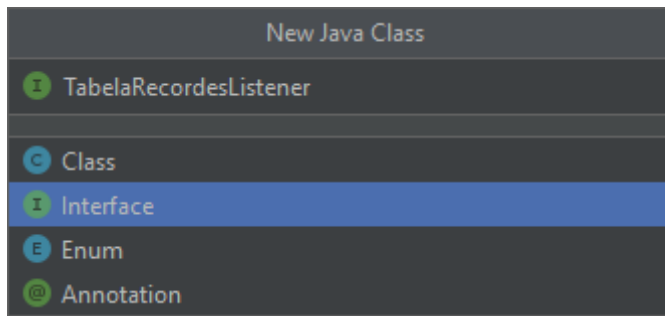
Atualização da Vista: Criação de Eventos

Neste momento, quando executar a aplicação e jogar e ganhar um jogo, reparará que o programa deteta se é um novo recorde para o nível de dificuldade em questão e pede-lhe o nome. No entanto, ao olhar para a tabela de recordes que desenhamos na janela principal, verificará que os dados lá contidos não foram atualizados. Vamos nesta secção atualizá-los corretamente.

A janela principal já contém os componentes que compõem a vista da tabela de pontuações. No entanto, para manter a coerência entre os dados reais e os dados apresentados, esses componentes devem ser alterados sempre que a tabela de pontuações se altere. Uma das técnicas comuns e possíveis de ser usadas nesta situação, e mantendo a coerência com o padrão MVC, é a utilização de eventos. Desta vez, o evento não vai indicar algo que o utilizador tenha feito, mas sim que o modelo de dados foi alterado. Como esse evento não existe, vamos criar um evento do zero.

Um evento não é mais do que a invocação de um método conhecido e registado para ser invocado. Há várias formas de o fazer, mas em Java a forma tradicional consiste na utilização de interfaces. São essas interfaces que tornam o método conhecido. Essas interfaces terminam normalmente com o sufixo Listener, para indicar que representam métodos que serão implementados por quem estiver “à escuta” de determinados eventos. Vamos criar então a nossa própria interface com o nome TabelaRecordesListener:





Na interface criada, vamos apenas definir um método, o `recordesAtualizados()`, que será invocado sempre que a tabela de recordes for atualizada.

```
public interface TabelaRecordesListener {  
    public void recordesAtualizados(TabelaRecordes recordes);  
}
```

Agora que temos a interface, vamos alterar a `TabelaRecordes` para que esta “lance” o evento `recordesAtualizados`. Para isso, a `TabelaRecordes` precisa saber quem é que está à escuta do evento. Adicione um atributo para guardar os “ouvintes”, e métodos para adicionar e remover esses ouvintes da tabela de recordes.

```
...  
private ArrayList<TabelaRecordesListener> listeners;  
  
public TabelaRecordes() {  
    ...  
    listeners=new ArrayList<>();  
}  
...  
public void addTabelaRecordesListener(TabelaRecordesListener list) {  
    listeners.add(list);  
}  
  
public void removeTabelaRecordesListener(TabelaRecordesListener list) {  
    listeners.remove(list);  
}  
...
```

Para lançar o evento para todos os ouvintes registados, basta que cada vez que o recorde seja atualizado se invoque o método `recordesAtualizados()` de todos os `TabelaRecordesListener` registados. Para ajudar, podemos criar um método auxiliar: `notifyRecordesAtualizados()`. O prefixo `notify` é comum, mas não obrigatório, para os métodos auxiliares que emitem eventos.

```
private void notifyRecordesAtualizados() {  
    for (TabelaRecordesListener list:listeners)  
        list.recordesAtualizados(this);  
}
```

Finalmente, quando o método `setRecorde()` for invocado e o recorde for melhorado, podemos invocar este `notifyRecordesAtualizados()` e assim informar todos os ouvintes.

```

public void setRecorde(String nome, long tempo) {
    if (tempo < this.tempo) {
        this.nome = nome;
        this.tempo = tempo;
        notifyRecordesAtualizados();
    }
}

```

Com isto terminámos a parte da emissão do evento. Falta a outra parte do evento: o tratamento do evento. Para isso, na janela principal, registre os ouvintes para cada tabela de recordes (fácil, médio e difícil) no construtor.

```

public MinesFinder() {
    // . . .

    recordesFacil = new TabelaRecordes();
    recordesMedio = new TabelaRecordes();
    recordesDificil = new TabelaRecordes();

    recordesFacil.addTabelaRecordesListener(new TabelaRecordesListener() {
        @Override
        public void recordesAtualizados(TabelaRecordes recordes) {
            recordesFacilAtualizado(recordes);
        }
    });

    recordesMedio.addTabelaRecordesListener(new TabelaRecordesListener() {
        @Override
        public void recordesAtualizados(TabelaRecordes recordes) {
            recordesMedioAtualizado(recordes);
        }
    });

    recordesDificil.addTabelaRecordesListener(new TabelaRecordesListener() {
        @Override
        public void recordesAtualizados(TabelaRecordes recordes) {
            recordesDificilAtualizado(recordes);
        }
    });
}

```

Com esta alteração, indicou que os métodos `recordes*Atualizado()` serão invocados quando os respetivos recordes forem alterados. Na implementação desses métodos (que deverá criar) bastará atualizar o texto das respetivas *labels*.


```

private void recordesFacilAtualizado(TabelaRecordes recordes) {
    lblNomeFacil.setText(recordes.getNome());
    lblTempoFacil.setText(Long.toString(recordes.getTempo()/1000));
}

private void recordesMedioAtualizado(TabelaRecordes recordes) {
    lblNomeMedio.setText(recordes.getNome());
    lblTempoMedio.setText(Long.toString(recordes.getTempo()/1000));
}

private void recordesDifícilAtualizado(TabelaRecordes recordes) {
    lblNomeDifícil.setText(recordes.getNome());
    lblTempoDifícil.setText(Long.toString(recordes.getTempo()/1000));
}

```

Se executar agora o jogo, já verá a tabela de recordes a atualizar-se corretamente.

Persistência: Ler e Gravar em Ficheiros Binários

Apesar dos recordes se atualizarem corretamente, cada vez que executa a aplicação, os dados introduzidos anteriormente perdem-se. Para resolver este problema, teremos que persistir (guardar) os dados em disco. Felizmente, a persistência de dados em ficheiros binários é muito simples de se fazer em Java.

O que se pretende guardar em disco são apenas as tabelas de recordes para os níveis fácil, médio e difícil. Ou seja, apenas os dados de uma classe. O Java consegue converter automaticamente os dados de uma classe para um ficheiro binário, desde que essa classe seja marcada como `Serializable`. Assim, marque a classe `TabelaRecordes` como `Serializable`. Marcar uma classe é dizer que esta implementa uma interface (sendo essa interface vazia, ou seja, não contendo métodos).

```

public class TabelaRecordes implements Serializable {
    private String nome;
    ...

```

No entanto, temos de ter em atenção que o Java tentará guardar os dados de todos os atributos, incluindo a lista de *listeners*! Essa lista não deveria ser guardada, portanto temos que informar o Java para não o guardar. Para indicar que atributos não devem ser guardados, temos que marcar esses atributos como *transient*, ou seja, temporários.

```

public class TabelaRecordes implements Serializable {
    private String nome;
    private long tempo;

    private transient ArrayList<TabelaRecordesListener> listeners;

```

Uma vez que este atributo é temporário, isso quer dizer que também não vai ser lido do ficheiro. O problema de não ser lido do ficheiro é que, quando lermos o ficheiro, a tabela de recordes vem com o `listeners` a `null` e não passará pelo construtor para inicializá-lo. Assim, teremos que ter outra abordagem para a inicialização deste atributo, assumindo sempre que ele pode estar a `null` em qualquer momento. Vamos então alterar todos os métodos que acedem a este atributo para que verifiquem se ele está a `null` ou não e ajam em conformidade.

```

public void addTabelaRecordesListener(TabelaRecordesListener list) {
    if (listeners == null) listeners = new ArrayList<>();
    listeners.add(list);
}

public void removeTabelaRecordesListener(TabelaRecordesListener list) {
    if (listeners != null) listeners.remove(list);
}

private void notifyRecordesAtualizados() {
    if (listeners != null) {
        for (TabelaRecordesListener list:listeners)
            list.recordesAtualizados(this);
    }
}
}

```

Agora que a classe TabelaRecordes está devidamente marcada, podemos passar à gravação em disco propriamente dita. Para isso, vamos acrescentar um método auxiliar na janela principal:

```

private void guardarRecordesDisco() {
    ObjectOutputStream oos = null;
    try {
        File f = new
File(System.getProperty("user.home")+File.separator+"minesfinder.recordes");
        oos = new ObjectOutputStream(new FileOutputStream(f));
        oos.writeObject(recordesFacil);
        oos.writeObject(recordesMedio);
        oos.writeObject(recordesDificil);
        oos.close();
    } catch (IOException ex) {
        Logger.getLogger(MinesFinder.class.getName()).log(Level.SEVERE, null,
ex);
    }
}
}

```

Este método guardará um ficheiro chamado “minesfinder.recordes” na pasta do utilizador (no Windows será tradicionalmente em “C:\Users\<utilizador>\minesfinder.recordes”, em Linux será “/home/<utilizador>/minesfinder.recordes”). NOTA: existem bugs no Java que fazem com que o System.getProperty(“user.home”) nem sempre funcione bem em Windows (nos casos em que a diretoria *home* seja gerida por um servidor remoto), mas vamos ignorar esse problema para melhorar a legibilidade do código.

O que o código acima faz é simplesmente escrever objetos (ObjectOutputStream) em ficheiros binários (FileOutputStream). Para fazer o reverso, ler objetos, utilizaremos um ObjectInputStream e um FileInputStream:

```

private void lerRecordesDoDisco() {
    ObjectInputStream ois = null;
    File f = new
File(System.getProperty("user.home")+File.separator+"minesfinder.recordes");
    if (f.canRead()) {
        try {
            ois = new ObjectInputStream(new FileInputStream(f));
            recordesFacil=(TabelaRecordes) ois.readObject();
            recordesMedio=(TabelaRecordes) ois.readObject();
            recordesDificil=(TabelaRecordes) ois.readObject();
            ois.close();
        } catch (IOException ex) {
            Logger.getLogger(MinesFinder.class.getName()).log(Level.SEVERE,
null, ex);
        } catch (ClassNotFoundException ex) {
            Logger.getLogger(MinesFinder.class.getName()).log(Level.SEVERE,
null, ex);
        }
    }
}
}

```

Terminados estes ficheiros auxiliares, invoque o `guardarRecordesDisco()` sempre que atualizar as *labels* das tabelas de recordes. Altere também o construtor para ler os recordes do disco ao iniciar o jogo. Não se esqueça de atualizar as *labels* logo após ler os recordes do disco.

```

...
recordesFacil = new TabelaRecordes();
recordesMedio = new TabelaRecordes();
recordesDificil = new TabelaRecordes();
lerRecordesDoDisco();

lblNomeFacil.setText(recordesFacil.getNome());
lblTempoFacil.setText(Long.toString(recordesFacil.getTempo()/1000));
lblNomeMedio.setText(recordesMedio.getNome());
lblTempoMedio.setText(Long.toString(recordesMedio.getTempo()/1000));
lblNomeDificil.setText(recordesDificil.getNome());
lblTempoDificil.setText(Long.toString(recordesDificil.getTempo()/1000));

recordesFacil.addTabelaRecordesListener(new TabelaRecordesListener() {
    @Override
    public void recordesAtualizados(TabelaRecordes recordes) {
        ...
    }
}

```

Conclusão

Este tutorial demonstrou os passos a serem seguidos na criação de um jogo completo. Este tutorial permitiu-nos criar uma aplicação gráfica interativa, utilizando componentes visuais de forma estática (editor visual) e dinâmica (em tempo de execução). Permitiu-nos também criar e personalizar componentes visuais com os botões do campo minado e utilizar eventos de forma estática e dinâmica. Para guardar as pontuações, criámos também eventos próprios e persistimos objetos em ficheiros binários. Esperamos que este tutorial tenha servido para conhecer o padrão *Model-View-Controller* e incentivar o seu uso e para relembrar conceitos básicos de P.O.O. e de Java.

Documentação / Links de interesse:

Sobre Layout em Java Swing:

<https://docs.oracle.com/javase/tutorial/uiswing/layout/visual.html>

<https://docs.oracle.com/javase/tutorial/uiswing/layout/layoutlist.html>

Como utilizar o GUI Designer do IntelliJ IDEA Ultimate:

<https://www.jetbrains.com/help/idea/gui-designer-basics.html>

Outros tutoriais de potencial interesse:

<https://www.javatpoint.com/java-layout-manager>

https://www.tutorialspoint.com/swing/swing_layouts.htm